# Canonical two-chunk planner user guide

Version 0.2.0. `two_chunk_planner` is a completely standalone, read-only canonical two-chunk planning tool. After installation it runs from any directory and needs no ULVZ project, SPECFEM worktree, or patch manifest.

## 1. Scope and limits

It plans only `NCHUNKS=2`, canonical 90°×90° geometry: AB is the central first chunk and AC is the supported-left second chunk. It does not support other widths, attachment sides, independent orientations, arbitrary two-chunk or multi-chunk topology. `general_two_chunk_mode_classification=B`.

The tool searches center latitude/longitude and `GAMMA_ROTATION_AZIMUTH`, classifies sources/receivers, audits paths and emits a reviewable parameter fragment. It never runs or replaces topology, mesher, database, Stacey, solver, waveform, or production boundary-return acceptance.

## 2. Install and start

Install a supplied wheel from any directory:

```
python -m pip install two_chunk_planner-0.2.0-py3-none-any.whl
```

Install from a source distribution or source directory:

```
python -m pip install two_chunk_planner-0.2.0.tar.gz
python -m pip install .
python -m pip install -e '.[phase-aware]'
```

The optional `phase-aware` extra provides ObsPy. Confirm geographic TauP support instead of assuming it:

```
python -c 'from obspy.geodetics.base import HAS_GEOGRAPHICLIB; print(HAS_GEOGRAPHICLIB)'
two-chunk-planner --help
python -m two_chunk_planner --help
```

For package development only, run without installing from the repository root:

```
PYTHONPATH=packages/two_chunk_planner/src /usr/bin/python3 -m two_chunk_planner --help
```

The installed `two-chunk-planner` command and the zero-install `python -m two_chunk_planner` form run the same planner. The latter is useful when developing this repository; it is not a second planner mode.

## 3. Quick standalone cases

The following fixtures are synthetic and not Kim/Song inputs. Set `P` to a copy of the package examples; every output directory must not exist.

```
P=/path/to/two_chunk_planner/examples
two-chunk-planner plan --cmtsolution "$P/geometry_only/DATA/CMTSOLUTION" \
  --stations "$P/geometry_only/DATA/STATIONS" --analysis-window 0 1900 \
  --latitude-range 0,0 --longitude-range 0,0 --gamma-range 0,0 --output geometry_plan
```

```
two-chunk-planner plan --cmtsolution "$P/phase_aware/DATA/CMTSOLUTION" \
  --stations "$P/phase_aware/DATA/STATIONS" --path-mode phase-aware \
  --phases Pdiff,Sdiff --taup-model prem --taup-resample --analysis-window 0 1900 \
  --latitude-range 0,0 --longitude-range 0,0 --gamma-range 0,0 --output phase_plan
```

```
two-chunk-planner plan --cmtsolution "$P/geometry_only/DATA/CMTSOLUTION" \
  --stations "$P/geometry_only/DATA/STATIONS" --par-file "$P/external_par_file/Par_file" \
  --analysis-window 0 1900 --latitude-range 0,0 --longitude-range 0,0 --gamma-range 0,0 \
  --output external_par_plan
```

## 4. Inputs and profile

Provide exactly one of `--cmtsolution` or `--source LAT LON DEPTH_KM`, and one of `--stations` or `--stations-csv`. Source, station and target YAML files may be at any readable path. Circle/polygon/corridor targets are optional.

`--par-file` is the only optional external configuration context. It may be at any path; it reads NEX/NPROC and relevant compatibility flags but is neither located through SPECFEM nor edited. If absent, planning defaults come from `src/two_chunk_planner/resources/canonical_profile_v1.json`: NEX=96 and the canonical geometry/provenance reference. These are planning defaults, not the user's production configuration. The profile never accesses a user SPECFEM tree, patch manifest, or source

hash.

The English CLI reference and Chinese CLI reference are the complete, authoritative 35-option list. Use them for TauP, target, search, scoring, NEX/MPI and output details; this guide explains only common workflow choices.

## 5. Event-style phase-aware command

Use a new output directory for every run. This portable form is equivalent to the Event 1 acceptance command, but intentionally uses placeholder paths:

```
D=/path/to/DATA
OUT=/path/to/planner_output

PYTHONPATH=packages/two_chunk_planner/src \
/usr/bin/python3 -m two_chunk_planner plan \
  --cmtsolution "$D/CMTSOLUTION" \
  --stations "$D/STATIONS" \
  --par-file "$D/Par_file" \
  --path-mode phase-aware \
  --phases S,Sdiff \
  --allow-partial-phase-coverage \
  --taup-model prem \
  --taup-resample \
  --analysis-window 0 1600 \
  --output "$OUT"
```

There must be no space after a shell continuation backslash. `--par-file` is optional and read only: it supplies NEX/NPROC and compatibility context for resource suggestions, but it does not change the geometry search, locate a SPECFEM checkout, verify a patch, or edit the file. `--analysis-window` gives the advisory boundary-time comparison its end time; it is not a production boundary-safety proof.

`--taup-resample` requests the program's documented TauP-path resampling and records that choice in the path metadata. It does not establish waveform accuracy. With `--allow-partial-phase-coverage`, unavailable requested station-phase pairs are recorded and not substituted. For example, missing Sdiff arrivals remain missing; available S paths can still be planned and the output contains the requested/provided/missing inventory. Without that flag, all missing pairs are reported and planning stops.

## 6. How planning works

`geometry-only` is the default and uses sampled surface great circles. `phase-aware` requests each requested phase×station with TauP; Pdiff/Sdiff geographic paths are validated. Strict coverage is default; partial coverage requires `--allow-partial-phase-coverage` and never substitutes a phase. TauP length is `taup_raypath_polyline_estimate`; CMB-near information is a sampled proxy. TauP is forbidden for boundary-return timing.

The planner reads the source, stations, optional target and phase paths; then it searches a canonical two-chunk location and orientation in deterministic coarse, local and final stages. Every candidate checks source, station, target and path coverage; external-boundary and C1/C2 endpoint margins contribute to the transparent score. A final resource pass checks NEX/NPROC compatibility and suggests compatible decompositions.

TauP paths are prepared before orientation search, once for each requested source/station/phase/model/resampling combination. The planner stores their global unit vectors in continuous NumPy arrays, rotates fixed chunk geometry for small orientation batches, and evaluates path coverage and finite-arc distances with blocked numerical operations. Search candidates keep only the compact feasibility, rejection, score and ordering data needed for the three stages. Conservative numerical guards trigger an exact scalar review for keeper candidates when needed; the chosen candidate always receives the full scalar `GeometryCandidate` audit used for JSON output. These implementation details are automatic: they do not remove candidates, stations, phases or path points, and they do not change score, stable sorting or tie-breaking.

NEX is not an orientation-search control. It and NPROC affect only the later compatibility/resource recommendation. NEX=96 at total ranks 2/8/12 is labelled `project_validated`; other compatible choices are not project-validated.

## 7. Planner output directory

Every successful `plan` run writes the following common files. They are ordinary planner outputs, not performance-validation artifacts.

| File | Type and purpose | Usually inspect |
|---|---|---|
| `report.md` | Short human-readable planning result. | Returned count, recommended center/gamma, score and phase inventory. |
| `candidates.json` | Structured search result with up to five ranked feasible candidates, search counts, rejection summary, phase inventory and path records. | `search`, `candidates`, `phase_inventory`, `phase_paths`. |
| `candidates.csv` | Flat summary of returned candidates. | Candidate coordinates, score and feasibility. |
| `geometry_audit.json` | Full scalar audit for the chosen candidate (or `chosen_candidate: null`). | Source/station classifications, `path_audits`, target audit, margins, score, warnings and rejection reasons. |
| `boundary_time_audit.json` | Advisory surface-arc boundary-time proxy. | `global_status`, records and margin to `analysis_end_s`; never treat it as a hard safety proof. |
| `recommended_Par_file.inc` | Review-only canonical geometry and resource fragment. | NCHUNKS, widths, center, gamma and any NEX/NPROC recommendation before manually transferring them to a Par_file. |
| `run_manifest.json` | Input/run provenance and mode metadata. | Source, stations, path mode, TauP settings, profile and Par_file provenance. |
| `map.png` | Latitude/longitude map. | Source, stations, paths, target, outer arcs and AB--AC interface. |
| `globe.png` | Three-dimensional globe view. | The closed spherical relationship of source, stations, paths and both chunks. |

The file set is the same for geometry-only and phase-aware runs. In geometry-only mode, `run_manifest.json` stores null TauP settings and `phase_inventory` has no requested TauP pairs; `phase_paths` contains the sampled surface geometry paths. In phase-aware mode, the manifest stores TauP model/resampling settings, `phase_inventory` records requested/provided/missing pairs, and `phase_paths` contains returned geographic TauP paths. `report.md` adds a phase-inventory section in phase-aware mode.

The final geometry is described by `center_latitude_deg`, `center_longitude_deg` and `gamma_rotation_azimuth_deg`; together with the fixed canonical 90° widths they define both chunks. AB is central and AC is supported-left: AC has no independent position or rotation. `source`, `stations` and `path_audits` report chunk classification, coverage and minimum external/C1/C2 distances. `score_components` exposes coverage, external and endpoint margins, cost proxy, normalized values and weights. `warnings` and `rejection_reasons` must be reviewed rather than ignored.

`map.png` is a longitude-latitude projection. A spherical closed outer arc that crosses ±180° is deliberately split there rather than joined by a false map-wide chord; high-latitude and great-circle arcs can also look strongly curved or distorted in this projection. Use `globe.png` to inspect the same boundaries as closed spherical geometry.

All audits record `planner_mode=standalone`, `compatibility_profile_version`, `specfem_source_verified=false`, `accepted_patch_verified=false`, `production_configuration_verified=false`, `par_file_source`, `configuration_status`, and `verification_warnings`. These false values are not planning failures: independent planning deliberately does not inspect a user's SPECFEM source, installed patch, mesh, or runtime validation.

`recommended_Par_file.inc` is a suggested parameter fragment, not a complete or runnable Par_file. Review candidates, geometry audit, boundary-time audit, report and fragment before copying anything manually. Files such as `summary.json`, `performance_matrix.csv`, profiles and persistent shell logs belong only to dedicated performance-validation directories; `plan` never creates them.

## 8. How to read a result quickly

1. Read `report.md` and `candidates.json`. A positive returned count and a feasible first candidate mean a layout was found.
2. Inspect `geometry_audit.json`: source and every required station should be in-domain; every required path should meet its coverage threshold; external boundary and C1/C2 margins should be suitable for the scientific review.
3. Read `phase_inventory` and `warnings`. Partial coverage is explicit; a missing Sdiff is not silently replaced by S.
4. Review `recommended_Par_file.inc` with the original `Par_file`. Confirm NEX/NPROC multiple and divisibility requirements, and distinguish values read from the external Par_file from planner suggestions.
5. Inspect both figures, then complete separate SPECFEM mesh, topology, Stacey, solver, waveform and boundary-return validation.

## 9. Complete workflow

1. Install the package and prepare source/stations.
2. Choose geometry-only screening or phase-aware review.
3. Optionally prepare a strict target YAML and/or external Par_file.
4. Run `plan` to a new output directory.
5. Review candidate ranking, classifications, margins, warnings and resource labels.
6. In a separate SPECFEM workflow, apply/verify the accepted patch, then validate mesh/topology, Stacey, solver, waveform and external returns.

## 10. Validation, performance and scientific status

Boundary seconds are always advisory `heuristic_not_conservative` surface-arc proxies; `boundary_time_production_safe=false`. Sampling

stability is `indeterminate` (Pdiff 0.130094%, Sdiff 0.076131% resample-length differences). Kim/Song exact reconstruction remains unavailable without author inputs.

This package is GPL-3.0-or-later; see third-party notices and validation status.

The preserved Event 1 phase-aware acceptance completed twice in 12:31.62 and 12:28.80, with strictly identical outputs and 20 package tests passing. This was about 32% faster than the preceding approximately 18.5-minute run, while the old implementation has only a >1800 s timeout lower bound. The 10-minute goal was not met. Runtime depends mainly on orientation candidates, returned phase paths and path sampling points, not on NEX alone. See the detailed Event 1 performance record.